

PLANO DE RE

GO-LIVE PENDENTE · 0/6 requisitos prontos · 2

Core \$29

Pro \$99

Enterprise \$499

DATA · 25/08/2026, 19:07:3

www.trinnityviseronssystem.io

TVS v5.0

1

Estado atual dos requisitos (ao vivo)

- Stripe (cobranças reais) — modo manual/dev — Checkout de subscrições e faturação em produção
- Webhook de pagamento — sem segredo de webhook — Endpoint /api/billing/webhook assinado para upgrade automático do plano
- Gmail OAuth (email real) — OAuth pendente — Agente de atendimento envia emails de verificação, faturas e respostas
- Email provider de produção — dev — Transporte de email ativo (dev/SMTP/Resend/SendGrid/Gmail)
- Domínio próprio — sem TVS_PUBLIC_URL — Domínio registado com TVS_PUBLIC_URL definido (<https://www.trinnityviseron.com>) para converter vendas
- Base de dados de produção — JSON fallback — Postgres para contas/faturas persistidas de forma robusta

Em falta:

processor · processor_webhook · gmail · email_provider · domain · database

2

Passo a passo para faturar

1. Stripe — ativar cobranças reais

Sem chave, o checkout cria sessões de demonstração (trial). Com STRIPE_SECRET_KEY de produção, o Stripe cobre subscrições e faturação.

- Criar conta em dashboard.stripe.com
- Settings !' Business !' completar dados da empresa (nome, NIF/VAT, endereço)
- Developers !' API keys !' revelar chave secreta (sk_live_...)
- Definir STRIPE_SECRET_KEY e STRIPE_WEBHOOK_SECRET no Render (.env)
- Criar 3 produtos/preços no Stripe: Core 29€, Pro 99€, Enterprise 499€/mês
- Guardar os price ids nos plans (src/web/billing/plans.ts priceId)

2. Stripe Webhook — upgrade automático

O evento checkout.session.completed tem de chegar ao servidor para o tenant passar a Pro/Enterprise sem intervenção manual.

- Stripe dashboard !' Developers !' Webhooks !' Add endpoint
- URL: <https://viseron-web.onrender.com/api/billing/webhook>
- Eventos: checkout.session.completed, invoice.payment_failed, customer.subscription.deleted
- Copiar o signing secret (whsec_...) e definir STRIPE_WEBHOOK_SECRET
- Testar com o modo de teste do Stripe antes de ir para produção

3. Gmail OAuth — emails reais

O agente de atendimento envia emails de boas-vindas, verificação, reposição de password e faturas.

- Google Cloud Console !' criar projeto 'Viseron'

- % APIs & Services !' Enable Gmail API
- % OAuth consent screen !' configurar app (test mode), adicionar contas de teste
- % Credentials !' OAuth 2.0 Client ID (Web) com redirect para /api/email/gmail/callback
- % Definir GMAIL_CLIENT_ID, GMAIL_CLIENT_SECRET, GMAIL_USER
- % Correr npm run gmail:setup !' autorizar !' guardar o GMAIL_REFRESH_TOKEN

4. Email provider em produção

Sem provider, os emails caem no modo dev (ficheiros). SMTP/Resend/SendGrid entregam emails reais.

- % Escolher um provider (Resend é o mais simples; SendGrid bom volume)
- % Criar chave API + verificar o domínio no provider
- % Definir EMAIL_PROVIDER=resend|sendgrid|smtp e a chave correspondente
- % Definir EMAIL_FROM=Trinnity Viseron System <no-reply@trinnityviseronssystem.io>

5. Domínio próprio + HTTPS

www.trinnityviseronssystem.io dá credibilidade ao checkout e às faturas. Sem ele, o sucesso da compra cai num domínio Render.

- % Registrar o domínio (Cloudflare ~10€/ano, Namecheap, GoDaddy)
- % Apontar DNS: CNAME/ALIAS para o Render (ou usar Vercel na frente)
- % Definir TVS_PUBLIC_URL=https://www.trinnityviseronssystem.io
- % npm run deploy:domain:check para validar DNS/HTTPS
- % Configurar SSL (Render/Vercel emitem certificado automático)

6. Base de dados Postgres

Contas e faturas em ficheiros JSON funcionam, mas Postgres dá robustez, concorrência e backups.

- % Criar Postgres grátis (Neon, Supabase) ou pago (Render, Railway)
- % Definir DATABASE_URL=postgres://...
- % npm run backup continua a arquivar os dados

3 Modelo de receita

Preços: Core \$29 · Pro \$99 · Enterprise \$499/mês (trial 14 dias em todos)

Conversão checkout !' cliente: Meta: 2-5% dos visitantes que iniciam o checkout

LTV médio estimado: Pro: 99€ × retenção ~12 meses "H 1.188€ por cliente

MRR a 30 clientes Pro: 99€ × 30 = 2.970€/mês

MRR a 100 clientes (mix): "H 7.000€/mês (recurring, estável)

Meta a 12 meses: 100-200 clientes pagantes · MRR 7-14k€/mês · ARR ~100k€

Estratégia de arranque: 10 primeiros clientes via contacto direto + landing + JARVIS; depois canal de conteúdo automático

4

Próximo passo imediato

Começar por: Stripe (cobranças reais)

Ação: Criar conta em stripe.com ! ' Developers ! ' API keys ! ' definir STRIPE_SECRET_KEY (sk_live_...)